

Web Application Development Practical - CRUD APIs using Node.js, Express.js and MongoDB

Aim: To create four APIs using Node.js, Express.js and MongoDB for CRUD Operations.

Theory:

CRUD stands for Create, Read, Update and Delete operations. Node.js is used for server-side programming, Express.js is a web framework for handling routes and APIs, and MongoDB is a NoSQL database used to store application data. REST APIs are used for communication between frontend and backend.

Requirements:

1. Node.js Installed
2. MongoDB Installed or MongoDB Atlas
3. VS Code Editor
4. Postman for API Testing

Detailed Procedure

Step 1: Create a project folder named crud-api.

Step 2: Open terminal inside project folder.

Step 3: Initialize Node project using `npm init -y` command.

Step 4: Install required packages.

Step 5: Create `server.js` file.

Step 6: Connect MongoDB database.

Step 7: Create schema and model.

Step 8: Create CRUD API routes.

Step 9: Run the server.

Step 10: Test APIs using Postman.

Installation Commands

```
npm init -y
npm install express mongoose body-parser nodemon
```

Project Structure

```
crud-api/
├──
├── server.js
├── package.json
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
```

```

const bodyParser = require('body-parser');
const app = express();
app.use(bodyParser.json());

mongoose.connect('mongodb://127.0.0.1:27017/studentDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

console.log("MongoDB Connected");

const studentSchema = new mongoose.Schema({
  name: String,
  email: String,
  course: String
});

const Student = mongoose.model('Student', studentSchema);

app.get('/', (req, res) => {
  res.send("CRUD API Running");
});

// CREATE API
app.post('/addStudent', async (req, res) => {
  const student = new Student({
    name: req.body.name,
    email: req.body.email,
    course: req.body.course
  });

  await student.save();

  res.send("Student Added Successfully");
});

// READ API
app.get('/students', async (req, res) => {
  const students = await Student.find();

  res.json(students);
});

// UPDATE API
app.put('/updateStudent/:id', async (req, res) => {
  await Student.findByIdAndUpdate(req.params.id, {
    name: req.body.name,
    email: req.body.email,
    course: req.body.course
  });

  res.send("Student Updated Successfully");
});

// DELETE API
app.delete('/deleteStudent/:id', async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);

  res.send("Student Deleted Successfully");
});

```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

CRUD APIs

```
1. Create API  
POST http://localhost:3000/addStudent  
  
2. Read API  
GET http://localhost:3000/students  
  
3. Update API  
PUT http://localhost:3000/updateStudent/:id  
  
4. Delete API  
DELETE http://localhost:3000/deleteStudent/:id
```

Sample JSON Data

```
{  
  "name": "Akash",  
  "email": "akash@gmail.com",  
  "course": "Computer Engineering"  
}
```

Run Server

```
node server.js
```

Expected Output

```
MongoDB Connected  
  
Server running on port 3000
```

Advantages

1. Fast API Development
2. Flexible NoSQL Database
3. Scalable Backend Architecture
4. Easy Integration with Frontend
5. Efficient Data Management

Explanation:

Express.js handles API routing while MongoDB stores student data. Mongoose is used to create schema and interact with MongoDB database. POST API inserts data, GET API retrieves data, PUT API updates existing data and DELETE API removes data from database.

Result: Four CRUD APIs were created successfully using Node.js, Express.js and MongoDB.